

LABORATORIO 2: ENTRADAS Y SALIDAS DE RASPBERRY PI

Prerrequisito: Sistema Operativo Raspbian

Parte 1: Salida digital en Raspberry Pi

Conceptos teóricos:

La Raspberry Pi es un SoC (Sistema en Chip) que se puede utilizar para desarrollar una gran variedad de sistemas. Por esta razón, es importante comprender su funcionamiento y aprender a gestionar las entradas y salidas de este sistema.

La Raspberry Pi requerirá una pantalla para visualizar todo el trabajo realizado. En este laboratorio aprenderemos a utilizar nuestras computadoras portátiles como monitor de la Raspberry y a trabajar mediante el protocolo de comunicación VNC.

Por otro lado, la Raspberry Pi Pico W también es un SoC que cuenta con un procesador de doble núcleo basado en ARM Cortex-M0+ y una frecuencia de 133 MHz. Este sistema embebido se puede programar utilizando un SDK en C/C++ o también mediante MicroPython.

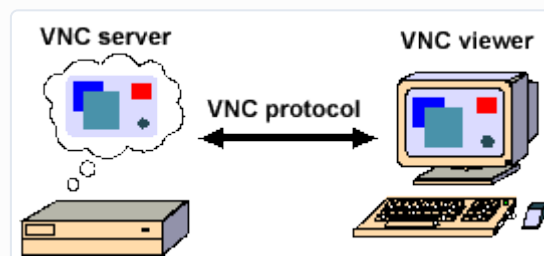
Instalación de VNC:

¿Qué es VNC?

Virtual Network Computing (Computación Gráfica en Red) es un sistema gráfico de compartir escritorios utilizado para controlar computadoras de forma remota. Este sistema permitirá a los usuarios visualizar la interfaz gráfica de un sistema específico desde otra fuente.

Para utilizar el visor VNC (VNC Viewer) necesitaremos conocer el usuario, la contraseña y la dirección IP de nuestra Raspberry Pi.

Podemos cambiar la contraseña utilizando el comando `passwd` en la terminal o en las preferencias de la Raspberry Pi.



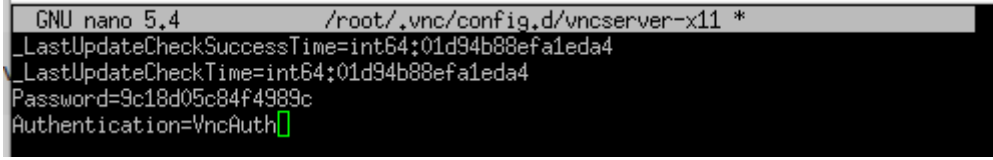
En la Raspberry:

- Habilitar VNC.
- Instalar el cliente Remmina en la computadora portátil con Ubuntu.

- Activar la contraseña de VNC con el comando: `sudo vncpasswd -service`
- Acceder mediante nano a la configuración del servidor VNC:

```
sudo nano /root/.vnc/config.d/vncserver-x11
```

Agregar la siguiente información al final del archivo:



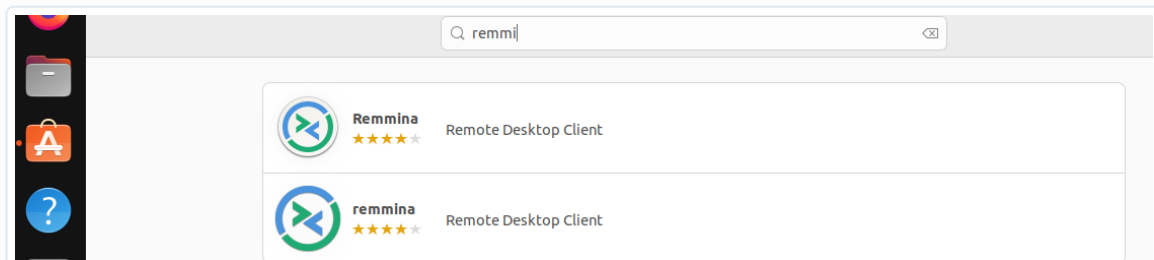
```
GNU nano 5.4 /root/.vnc/config.d/vncserver-x11 *
_LastUpdateCheckSuccessTime=int64:01d94b88efa1eda4
LastUpdateCheckTime=int64:01d94b88efa1eda4
Password=9c18d05c84f4989c
Authentication=VncAuth
```

```
GNU nano 5.4 /root/.vnc/config.d/vncserver-x11 *
_LastUpdateCheckSuccessTime=int64:01d94b88efa1eda4
LastUpdateCheckTime=int64:01d94b88efa1eda4
Password=9c18d05c84f4989c
Authentication=VncAuth
```

- Reiniciar VNC usando:

```
sudo systemctl restart vncserver-x11-serviced
```

Finalmente, debemos abrir y configurar Remmina:



Seleccionaremos el complemento VNC, introduciremos la dirección IP de la Raspberry, el usuario y la contraseña.

Como alternativa, se puede utilizar VNC Viewer en Ubuntu. Es un software de uso sencillo que ofrecerá características similares a Remmina sin demasiada configuración.

Para instalar VNC Viewer puedes seguir el siguiente enlace:

<https://www.realvnc.com/en/connect/download/viewer/linux/?laisr=10-14&laisl=>

O alternativamente puedes descargarlo en tu SO Ubuntu utilizando la tienda de aplicaciones de Ubuntu.

Comunicación SSH:

SSH requiere un nombre de host / dirección IP, un puerto, credenciales de usuario y contraseña.

Para utilizar SSH podemos seguir los siguientes comandos:

```
hostname -l

ssh [nombre_de_usuario]@[direccion_ip]
```

¿Qué puerto se utiliza por defecto?

La Raspberry utilizará el módulo GPIO para generar salidas y entradas dependiendo del código que generemos. Algunos conceptos importantes a considerar son:

- `GPIO.setwarnings(False)` deshabilitará las advertencias de GPIO, permitiéndonos trabajar sin inconvenientes iniciales.
- `GPIO.setmode()` configurará la numeración de los pines GPIO. ¿Cuántos modos tiene y cuáles son sus nombres?
- `GPIO.setup(pin, GPIO.type)` este método definirá un número de pin o definición como salida o entrada. ¿Cómo podemos usar este método?
- ¿Cómo podemos hacer que nuestro código funcione infinitamente? ¿Qué diferencias existen entre `for` y `while`?
- `GPIO.output(pin, GPIO.value)` genera un valor de salida, este puede ser HIGH (ALTO) o LOW (BAJO).
- `GPIO.input(pin_a_leer)` lee el valor de una entrada GPIO.

Probemos un ejemplo práctico:

```

# Definiciones de pines

led_pin = 12

# Suprimir advertencias

GPIO.setwarnings(False)

# Usar la numeración de pines "GPIO" (BCM)

GPIO.setmode(GPIO.BCM)

# Configurar el pin del LED como salida

GPIO.setup(led_pin, GPIO.OUT)

# Parpadear por siempre

while True:

    GPIO.output(led_pin, GPIO.HIGH) # Encender LED

    time.sleep(1)                  # Retraso de 1 segundo

    GPIO.output(led_pin, GPIO.LOW) # Apagar LED

    time.sleep(1)                  # Retraso de 1 segundo

```

¿Cómo podemos implementar un botón en el código anterior?

Ejercicio 1

Utilizando un botón y 2 LEDs, desarrolle un sistema que pueda modificar el estado de los LEDs. El sistema debe cumplir con lo siguiente:

- **Estado 1:** Parpadear ambos LEDs alternando la secuencia con intervalos de 1 segundo (cuando el LED 1 está encendido, el LED 2 está apagado).
- **Estado 2:** Ambos LEDs deben parpadear simultáneamente con intervalos de 2 segundos.
- **Estado 3:** En este estado, ambos LEDs deben permanecer ENCENDIDOS indefinidamente.
- **Estado 4:** Este estado apaga ambos LEDs.

El sistema debe regresar al estado 1 después del estado 4.

Modifique el código desarrollado para que funcione utilizando programación funcional.

Ejercicio 2

Utilizando 4 LEDs y 2 botones, realice un contador binario con los valores de la tabla 1. El contador debe tener las siguientes características:

- Cuando se presiona el botón 1, el contador binario aumenta 1 valor.
- Cuando se presiona el botón 2, el contador binario disminuye 1 valor.
- Cada LED representa los valores binarios.
- Cuando el contador alcanza el valor 15, debe regresar al primer valor (0).
- Cuando el contador está en 0, no debe seguir reduciendo el valor (debe permanecer en 0 después de presionar el botón).

Decimal	Binario	Hex
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Tabla 1. Valores de convención decimal, hexadecimal y binaria.

Finalmente, imprima en la consola el valor en 3 representaciones (hexadecimal, binario, decimal).

Modifique el código desarrollado para que funcione utilizando Programación Orientada a Objetos (POO).

Ejercicio 3

Tenemos un invernadero que necesita cultivar diferentes tipos de vegetales. Después de algunos estudios, el equipo determinó que la temperatura óptima para un buen cultivo va de 12 a 20 °C. Para mejorar el invernadero, el equipo decidió desarrollar un sistema de control de temperatura con las siguientes características:

- Encender el calefactor si el registro de temperatura está por debajo de 12 °C.
- Encender el ventilador cuando la temperatura supere los 20 °C.
- Apagar ambos dispositivos si la temperatura es óptima.

Para este sistema utilizaremos un LED rojo para simular el calefactor y un pequeño ventilador. Inicialmente, la temperatura debe ser generada por un número aleatorio. Después de eso, reemplace el generador de números aleatorios por un sensor DHT22 para la medición de temperatura.

Ejercicio 4

Genere un programa que encienda 1 de 4 LEDs posibles. Para lograr esto, el sistema tendrá 2 botones: el primero seleccionará el LED que se encenderá y el segundo aumentará 1 segundo al tiempo que el LED permanece encendido. Cuando se presione el primer botón, el LED a activar cambiará al siguiente y el tiempo de encendido se restablecerá a 1 segundo.

Implemente programación funcional.

Configuración de Raspberry Pi Pico W

Para trabajar con la tarjeta Pico W existen varias alternativas. La que requiere menor configuración es utilizar también Thonny y flashear la tarjeta mediante una conexión de comunicación serial (USB flash). La otra alternativa es trabajar utilizando VSCode. Para ello hay varios pasos a seguir y configurar el entorno.

Ejercicio 5

Asegúrese de haber instalado correctamente el software y/o las dependencias requeridas para trabajar adecuadamente con la Raspberry Pi y la Pico W. Asegúrese también de contar con las credenciales de git para trabajar en su entorno.